

ME 4030: Final Project

Worked Solutions

Quarter-Car Active Suspension
Corrected public edition, September 2026

Scope. This is a worked reference design for the corrected project handout, not a claim that all benchmarks can be met with one tuning. Equations, gains, plots, and tables use the same MATLAB model and `solutions/config.json`. The original project diagram is retained; controller plots are regenerated from the corrected MATLAB implementation. Historical plots are not used as numerical evidence.

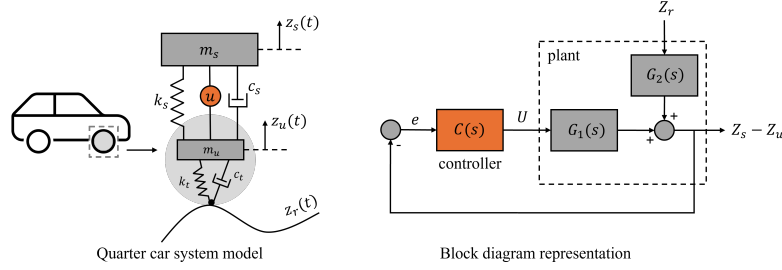


Figure 1: Original quarter-car diagram supplied with the project.

Parameter	Value	Meaning
M_s	2500 kg	Sprung mass
M_u	320 kg	Unsprung mass
K_s	80000 N/m	Suspension stiffness
C_s	350 N s/m	Suspension damping
K_t	500000 N/m	Tire stiffness
C_t	15020 N s/m	Tire damping

Model conventions

All displacements are perturbations about static equilibrium with one common positive direction. Let $x_s = z_s - z_u$ and $x_h = z_u - z_r$. The actuator exerts equal and opposite forces. Newton's law gives

$$M_s \ddot{z}_s = -K_s(z_s - z_u) - C_s(\dot{z}_s - \dot{z}_u) + u, \quad (1)$$

$$M_u \ddot{z}_u = K_s(z_s - z_u) + C_s(\dot{z}_s - \dot{z}_u) - K_t(z_u - z_r) - C_t(\dot{z}_u - \dot{z}_r) - u. \quad (2)$$

Positive tire deflection and its derivative create restoring terms on the wheel. Reversing the suspension output sign requires reversing the feedback convention; mixing the two definitions is an error.

Task 1(a,b): Transfer functions

With zero initial conditions and Laplace variable p , define

$$a(p) = M_s p^2 + C_s p + K_s, \quad b(p) = C_s p + K_s, \quad (3)$$

$$d(p) = M_u p^2 + (C_s + C_t)p + K_s + K_t, \quad e(p) = C_t p + K_t. \quad (4)$$

The transformed force balances are

$$\begin{bmatrix} a & -b \\ -b & d \end{bmatrix} \begin{bmatrix} Z_s \\ Z_u \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -1 & e \end{bmatrix} \begin{bmatrix} U \\ Z_r \end{bmatrix}. \quad (5)$$

The inverse of the dynamic-stiffness matrix is *not* itself the input/output transfer matrix. First invert, then multiply by the input-force matrix:

$$\Delta(p) = a(p)d(p) - b(p)^2, \quad (6)$$

$$\begin{bmatrix} Z_s \\ Z_u \end{bmatrix} = \frac{1}{\Delta} \begin{bmatrix} d & b \\ b & a \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -1 & e \end{bmatrix} \begin{bmatrix} U \\ Z_r \end{bmatrix} = \frac{1}{\Delta} \begin{bmatrix} d-b & be \\ b-a & ae \end{bmatrix} \begin{bmatrix} U \\ Z_r \end{bmatrix}. \quad (7)$$

Subtract the second output from the first:

$$\boxed{\begin{aligned} G_1(p) &= \left. \frac{X_s}{U} \right|_{Z_r=0} = \frac{(M_s + M_u)p^2 + C_t p + K_t}{\Delta(p)}, \\ G_2(p) &= \left. \frac{X_s}{Z_r} \right|_{U=0} = \frac{-M_s p^2 (C_t p + K_t)}{\Delta(p)}. \end{aligned}} \quad (8)$$

Expansion gives

$$\begin{aligned} \Delta(p) &= M_s M_u p^4 + [M_s(C_s + C_t) + M_u C_s]p^3 \\ &\quad + [M_s(K_s + K_t) + M_u K_s + C_s C_t]p^2 \\ &\quad + (C_s K_t + C_t K_s)p + K_s K_t, \end{aligned} \quad (10)$$

$$\Delta(p) = 800000p^4 + 38537000p^3 + 1480857000p^2 + 1376600000p + 40000000000, \quad (11)$$

$$G_1(p) = \frac{2820p^2 + 15020p + 500000}{\Delta(p)}, \quad G_2(p) = \frac{-37550000p^3 - 1250000000p^2}{\Delta(p)}. \quad (12)$$

Checks: $G_1(0) = 1/K_s$ (static actuator force stretches the suspension), $G_2(0) = 0$ (both masses eventually follow a constant road height), and both transfer functions are strictly proper.

Task 1(c): Passive response

Set $u = 0$ and use $X_s = G_2 Z_r$. For the delayed step, $Z_r(p) = 0.20e^{-p}/p$; for the sine, $Z_r(p) = 0.10\pi/(p^2 + \pi^2)$. Recover body and wheel motion from be/Δ and ae/Δ , respectively. The physical four-state poles are

$$-23.975782 \pm 35.186924i, \quad -0.109843 \pm 5.250445i. \quad (13)$$

All real parts are negative, but the lightly damped body mode explains the long passive oscillation. Its step response does not settle during the 10 s record.

Task 1(d): PID design and disturbance feedback

Use reference $r = 0$, error $e_c = r - x_s = -x_s$, and

$$\dot{\xi} = x_s, \quad u = -K_p x_s - K_d \dot{x}_s - K_i \xi, \quad (14)$$

$$C_{\text{PID}}(p) = K_p + K_d p + K_i/p. \quad (15)$$

Since $X_s = G_1 U + G_2 Z_r$ and $U = -C_{\text{PID}} X_s$,

$$\boxed{\frac{X_s}{Z_r} = \frac{G_2}{1 + G_1 C_{\text{PID}}}, \quad \frac{U}{Z_r} = \frac{-C_{\text{PID}} G_2}{1 + G_1 C_{\text{PID}}}.} \quad (16)$$

This differs from the reference-tracking transfer function $G_1 C_{\text{PID}}/(1 + G_1 C_{\text{PID}})$ displayed during tuning. Write $N_1 = (M_s + M_u)p^2 + C_t p + K_t$ and $N_2 = -M_s p^2 (C_t p + K_t)$. Then the disturbance response has denominator

$$D_c(p) = p\Delta(p) + (K_d p^2 + K_p p + K_i)N_1(p), \quad \frac{X_s}{Z_r} = \frac{pN_2(p)}{D_c(p)}. \quad (17)$$

Retaining the supplied example PID gains gives

$$K_p = 208000 \text{ N/m}, \quad K_d = 832000 \text{ N s/m}, \quad K_i = 624000 \text{ N/(m s)}. \quad (18)$$

These are an illustrative ideal-derivative design, not a feasible actuator specification. Its poles are approximately

$$-2974.911350, \quad -2.857898 \pm 12.904228i, \quad -0.172052 \pm 0.849037i. \quad (19)$$

The response is asymptotically stable; the fastest mode has a 0.336 ms time constant. A 1 ms output grid alone is inadequate for a trapezoidal force-energy integral.

Task 1(e,f): Step and sine interpretation

The integral of the wheel equation across an ideal step of height h yields

$$\Delta \dot{z}_u = \frac{C_t}{M_u} h = 9.3875 \text{ m/s}, \quad \Delta \dot{z}_s = 0, \quad \Delta \dot{x}_s = -9.3875 \text{ m/s}. \quad (20)$$

Positions and ξ stay continuous. The derivative feedback therefore produces

$$u(1^+) = K_d \frac{C_t}{M_u} h = 7,810,400 \text{ N}. \quad (21)$$

This finite jump is extreme; it is not an impulse in actuator force. The road derivative is impulsive because the road displacement is discontinuous. The PID keeps suspension travel small by forcing the body to follow the wheel, at the expense of body acceleration and force. The smooth sine avoids this step jump, but the same tuning still misses the acceleration benchmark. Numerical tables and corrected MATLAB plots follow after the LQR derivation.

Task 2(a): Physical state-space model

Let $q = [z_s, \dot{z}_s, z_u, \dot{z}_u]^T$. Rearranging the two force balances gives

$$\dot{q} = \bar{A}q + B_u u + B_{z_r} z_r + B_{\dot{z}_r} \dot{z}_r, \quad (22)$$

$$\bar{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -K_s/M_s & -C_s/M_s & K_s/M_s & C_s/M_s \\ 0 & 0 & 0 & 1 \\ K_s/M_u & C_s/M_u & -(K_s + K_t)/M_u & -(C_s + C_t)/M_u \end{bmatrix}, \quad (23)$$

$$B_u = \begin{bmatrix} 0 \\ 1/M_s \\ 0 \\ -1/M_u \end{bmatrix}, \quad B_{z_r} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ K_t/M_u \end{bmatrix}, \quad B_{\dot{z}_r} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ C_t/M_u \end{bmatrix}, \quad x_s = [1 \quad 0 \quad -1 \quad 0] q. \quad (24)$$

Task 2(b): Eliminate the road derivative

Define

$$\alpha = \frac{C_s}{M_s} + \frac{C_s + C_t}{M_u}, \quad \beta = \frac{K_s}{M_s} + \frac{K_s + K_t}{M_u}, \quad \gamma = \frac{1}{M_s} + \frac{1}{M_u}. \quad (25)$$

Subtracting wheel acceleration from body acceleration and using $z_u = z_s - x_s$, $\dot{z}_u = \dot{z}_s - \dot{x}_s$, gives

$$\ddot{x}_s = -\beta x_s - \alpha \dot{x}_s + \frac{K_t}{M_u} z_s + \frac{C_t}{M_u} \dot{z}_s + \gamma u - \frac{K_t}{M_u} z_r - \frac{C_t}{M_u} \dot{z}_r. \quad (26)$$

Choose a velocity-like coordinate

$$\eta = \dot{x}_s - \frac{C_t}{M_u} z_s + \alpha x_s + \frac{C_t}{M_u} z_r, \quad (27)$$

$$\dot{\eta} = \ddot{x}_s - \frac{C_t}{M_u} \dot{z}_s + \alpha \dot{x}_s + \frac{C_t}{M_u} \dot{z}_r = \frac{K_t}{M_u} z_s - \beta x_s + \gamma u - \frac{K_t}{M_u} z_r. \quad (28)$$

The road-derivative terms cancel exactly. Equivalently,

$$\eta(t) = \eta(0) + \int_0^t \left[\frac{K_t}{M_u} z_s - \beta x_s + \gamma u - \frac{K_t}{M_u} z_r \right] d\tau. \quad (29)$$

An auxiliary integral containing only control and road terms omits required state terms and is not equivalent to this plant. Also, η is not the integral error ξ added for controller design. Recover the original velocities by

$$\dot{x}_s = \frac{C_t}{M_u} z_s - \alpha x_s + \eta - \frac{C_t}{M_u} z_r, \quad (30)$$

$$\dot{z}_u = \dot{z}_s - \dot{x}_s. \quad (31)$$

At a road step the jumps in $\dot{x}_s$ and $C_t z_r/M_u$ cancel, so η is continuous.

Task 2(b), continued: Matrices and verification

Substitute the recovered $\dot{x}_s$ into $\ddot{z}_s = (-K_s x_s - C_s \dot{x}_s + u)/M_s$. With $x = [z_s, \dot{z}_s, x_s, \eta]^T$,

$$\dot{x} = Ax + B_1 u + B_2 z_r, \quad x_s = Cx, \quad (32)$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -C_s C_t / (M_s M_u) & 0 & C_s \alpha / M_s - K_s / M_s & -C_s / M_s \\ C_t / M_u & 0 & -\alpha & 1 \\ K_t / M_u & 0 & -\beta & 0 \end{bmatrix}, \quad (33)$$

$$B_1 = \begin{bmatrix} 0 \\ 1/M_s \\ 0 \\ \gamma \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 \\ C_s C_t / (M_s M_u) \\ -C_t / M_u \\ -K_t / M_u \end{bmatrix}, \quad C = [0 \ 0 \ 1 \ 0]. \quad (34)$$

Numerically, $\alpha = 48.17125$, $\beta = 1844.5$, and $\gamma = 0.003525$ in consistent SI units:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -6.57125 & 0 & -25.256025 & -0.14 \\ 46.9375 & 0 & -48.17125 & 1 \\ 1562.5 & 0 & -1844.5 & 0 \end{bmatrix}, \quad (35)$$

$$B_1 = \begin{bmatrix} 0 \\ 0.0004 \\ 0 \\ 0.003525 \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 \\ 6.57125 \\ -46.9375 \\ -1562.5 \end{bmatrix}. \quad (36)$$

The coordinate transformation is input dependent:

$$x = T_q q + d_r z_r, \quad T_q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & -1 & 0 \\ \alpha - C_t / M_u & 1 & -\alpha & -1 \end{bmatrix}, \quad d_r = \begin{bmatrix} 0 \\ 0 \\ 0 \\ C_t / M_u \end{bmatrix}. \quad (37)$$

Direct substitution verifies

$$A = T_q \bar{A} T_q^{-1}, \quad B_1 = T_q B_u, \quad (38)$$

$$B_2 = T_q B_{z_r} - A d_r, \quad 0 = T_q B_{\dot{z}_r} + d_r. \quad (39)$$

Thus the physical and transformed plants have the same poles, listed in Task 1(c). Both are stable. Controllability is tested using

$$\mathcal{C} = [B_1, AB_1, A^2 B_1, A^3 B_1], \quad \text{rank}(\mathcal{C}) = 4. \quad (40)$$

Independently, $C(pI - A)^{-1} B_1 = G_1(p)$ and $C(pI - A)^{-1} B_2 = G_2(p)$ agree with the force-balance transfer functions.

Task 2(c): Passive simulation

Set $u = 0$ and propagate this plant for the same two road signals. Recover $z_u = x_1 - x_3$, $\dot{z}_u = x_2 - \dot{x}_s$, and $\ddot{z}_s = (-K_s x_3 - C_s \dot{x}_s)/M_s$. These outputs match the physical plant; treating x_4 as wheel or relative velocity does not.

Task 2(d): Integral-augmented LQR design

For regulation of suspension deflection, use

$$x_a = \begin{bmatrix} x \\ \xi \end{bmatrix}, \quad \dot{\xi} = Cx, \quad (41)$$

$$\dot{x}_a = A_a x_a + B_a u + B_{ra} z_r, \quad A_a = \begin{bmatrix} A & 0 \\ C & 0 \end{bmatrix}, \quad B_a = \begin{bmatrix} B_1 \\ 0 \end{bmatrix}, \quad B_{ra} = \begin{bmatrix} B_2 \\ 0 \end{bmatrix}. \quad (42)$$

The augmented controllability matrix has rank 5. The unused integrator in an augmented *passive* simulation has a zero pole; this does not make the physical four-state plant unstable. For the nominal unforced regulator choose

$$J = \int_0^\infty (x_a^T Q x_a + u^T R u) dt, \quad u = -K x_a, \quad (43)$$

$$Q = \text{diag}(3 \times 10^7, 8 \times 10^7, 8 \times 10^7, 2 \times 10^7, 2 \times 10^{10}), \quad R = 0.01. \quad (44)$$

The weights penalize z_s , $\dot{z}_s$, x_s , η , and accumulated x_s , respectively. The fourth weight is on a transformed coordinate, not wheel velocity. Different state units make raw weight magnitudes incomparable without scaling. Increasing R discourages force; stronger travel or integral weighting generally trades comfort and effort for tighter regulation. These choices do not directly minimize body acceleration or impose hard force/travel limits.

To derive the gain, let $V = x_a^T P x_a$ and minimize

$$x_a^T Q x_a + u^T R u + 2x_a^T P (A_a x_a + B_a u). \quad (45)$$

Stationarity in u gives $2Ru + 2B_a^T P x_a = 0$. Substitution yields the algebraic Riccati equation and feedback

$$\boxed{A_a^T P + P A_a - P B_a R^{-1} B_a^T P + Q = 0,} \quad (46)$$

$$K = R^{-1} B_a^T P. \quad (47)$$

With the stabilizing solution P , MATLAB gives

$$K = \begin{bmatrix} 557537.198 & 108661.748 & -27934.083 & 33652.504 & 1414213.562 \end{bmatrix}. \quad (48)$$

The five closed-loop poles are

$$-149.992093, \quad -46.752838, \quad -8.339970, \quad -2.588062 \pm 2.900871i. \quad (49)$$

All are stable. For a constant road h , equilibrium has $\dot{\xi} = x_s = 0$; the force balances then imply $u = 0$ and $z_s = z_u = h$. In these coordinates $\eta = 0$ and the integral offset is $\xi = -K_1 h / K_5$. Integral augmentation enforces zero error at a stable feasible equilibrium; the passive plant already has zero static deflection for a constant road.

LQR optimality is for the nominal unforced regulation problem. It does not imply that the infinite-horizon cost remains finite with a persistent road offset or sine, or that this tuning is optimal for every disturbance/performance metric.

Tasks 1–2: Common closed-loop simulation

Implement both controllers as $u = -K_c x_a + F z_r$. For LQR, $K_c = K$ and $F = 0$. For the ideal PID, $CB_1 = 0$ and

$$\dot{x}_s = CAx + CB_2 z_r, \quad (50)$$

$$K_c = [K_p C + K_d CA \quad K_i], \quad F = -K_d CB_2, \quad (51)$$

$$A_c = A_a - B_a K_c, \quad B_c = B_{ra} + B_a F. \quad (52)$$

The feedthrough $F z_r$ is required for the PID law to differentiate the actual suspension deflection. Omitting it changes the controller. For passive operation use $K_c = 0$, $F = 0$, ignoring the unused integral state in stability discussions.

Exact continuous-input propagation. Use $\Delta t = 0.001$ s and $T = 10$ s. For each constant-road interval, set $z = [x_a^T, z_r]^T$ and

$$\dot{z} = H_s z, \quad H_s = \begin{bmatrix} A_c & B_c \\ 0 & 0 \end{bmatrix}, \quad z(t + \Delta t) = E z(t), \quad E = e^{H_s \Delta t}. \quad (53)$$

At $t = 1$ s set $z_r = 0.20$ while preserving x_a continuity. For the sine use $z = [x_a^T, z_r, \dot{z}_r]^T$, $\omega = \pi$, and

$$H_\omega = \begin{bmatrix} A_c & B_c & 0 \\ 0 & 0 & 1 \\ 0 & -\omega^2 & 0 \end{bmatrix}, \quad z(0) = \begin{bmatrix} 0_{5 \times 1} \\ 0 \\ 0.10\omega \end{bmatrix}. \quad (54)$$

This propagates a continuous sine instead of a stair-stepped approximation.

Exact RMS integration. For any linear output $y = h_y z$, its energy on one interval starting at z_k is

$$\int_0^{\Delta t} y(t_k + \tau)^2 d\tau = z_k^T W_y z_k, \quad (55)$$

$$W_y = \int_0^{\Delta t} e^{H^T \tau} h_y^T h_y e^{H \tau} d\tau. \quad (56)$$

Compute this finite-horizon Gramian with a block exponential:

$$\exp \left(\begin{bmatrix} -H^T & h_y^T h_y \\ 0 & H \end{bmatrix} \Delta t \right) = \begin{bmatrix} E^{-T} & V_{12} \\ 0 & E \end{bmatrix}, \quad W_y = E^T V_{12}. \quad (57)$$

Then $y_{\text{RMS}} = \sqrt{\sum_k z_k^T W_y z_k / T}$. Construct the rows from

$$u = -K_c x_a + F z_r, \quad x_h = x_1 - x_3 - z_r, \quad (58)$$

$$\ddot{z}_s = (-K_s x_3 - C_s \dot{x}_s + u) / M_s. \quad (59)$$

The implementation integrates acceleration, tire deflection, and force squared in this way. An independent adaptive ODE calculation and a halved output interval verify the RMS results. Reported peaks and settling times are sampled quantities; the ideal PID force jump is captured at onset, but sub-sample extrema elsewhere can require refinement. Rise time uses first threshold crossings on the same grid.

Tasks 1(e,f), 2(e,f): Numerical results

RMS values use the entire 10 s record, including the zero-input second before the step. Transient times are measured from the disturbance onset; rise time is the 10–90% crossing difference. Values below are generated from the same numerical model used by MATLAB, with independent agreement checks.

Step road

Metric	Passive	PID	LQR
a_{RMS} (m/s ²)	2.498	13.275	0.753
$x_{h,\text{RMS}}$ (m)	0.01364	0.01845	0.00613
$\max x_s $ (m)	0.22068	0.00625	0.19599
u_{RMS} (kN)	0.000	33.173	3.050
u_{peak} (kN)	0.000	7810.400	33.314
Body overshoot (%)	95.35	54.31	3.21
Body settling (s)	not settled	1.270	1.621
Suspension recovery (s)	not settled	0.142	1.618
Body rise, 10–90% (s)	0.190	0.081	0.749

Sinusoidal road

Metric	Passive	PID	LQR
a_{RMS} (m/s ²)	1.593	0.860	0.450
$x_{h,\text{RMS}}$ (m)	0.00830	0.00474	0.00242
$\max x_s $ (m)	0.11755	0.00129	0.10437
u_{RMS} (kN)	0.000	2.141	5.384
u_{peak} (kN)	0.000	8.150	7.880

“Not settled” means that the 2% band is not maintained through the end of this record, not that the system is unstable. Step definitions do not apply to the sine. **Benchmark assessment.** In the step run, passive control fails comfort, travel, and recovery, while tire RMS is acceptable. PID has excellent travel and recovery, acceptable tire RMS, but fails acceleration and RMS force. LQR has excellent tire RMS and recovery and acceptable RMS force, but fails acceleration ($0.753 > 0.70$) and travel ($0.196 > 0.040$). Its body overshoot is good under the supplemental 5% criterion; PID and passive fail the 10% criterion.

For the sine, PID has excellent travel and tire RMS and good RMS force, but fails comfort. LQR has good comfort and excellent tire RMS but fails travel and RMS force. Passive operation fails comfort and travel; tire RMS is good. Zero passive actuator force meets the force benchmark but does not establish a good overall suspension.

Corrected MATLAB figures: road step

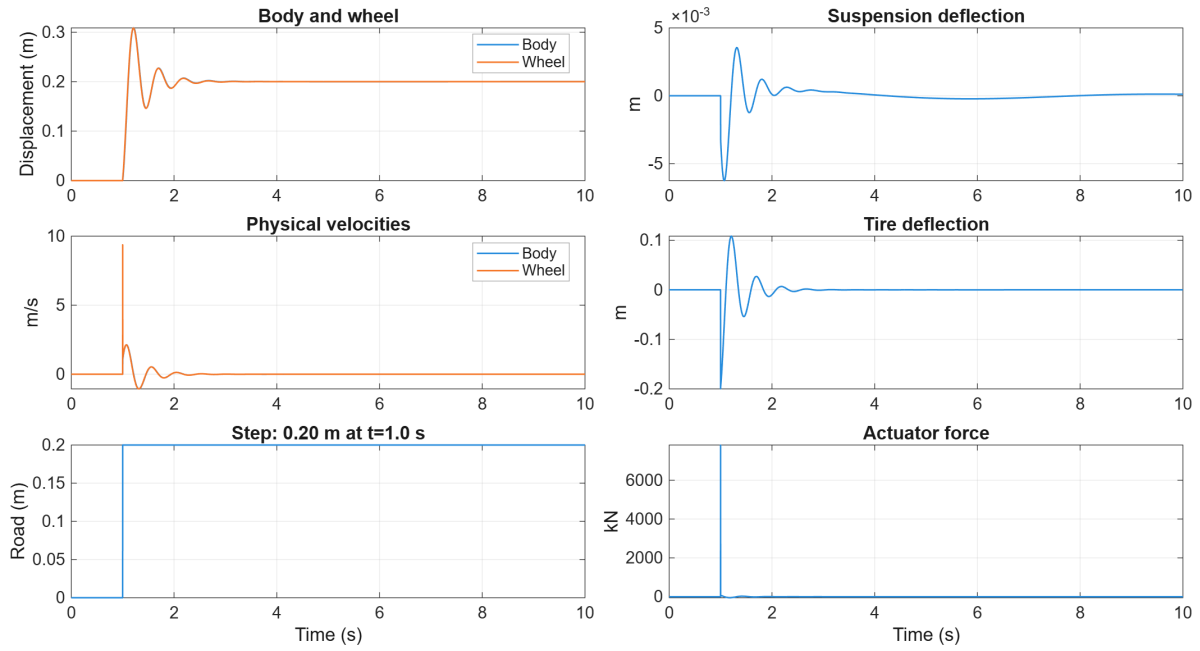


Figure 2: PID step response, regenerated in MATLAB using the original six-panel layout. The force peak is 7810.4 kN; the extreme initial transient is an ideal-model result.

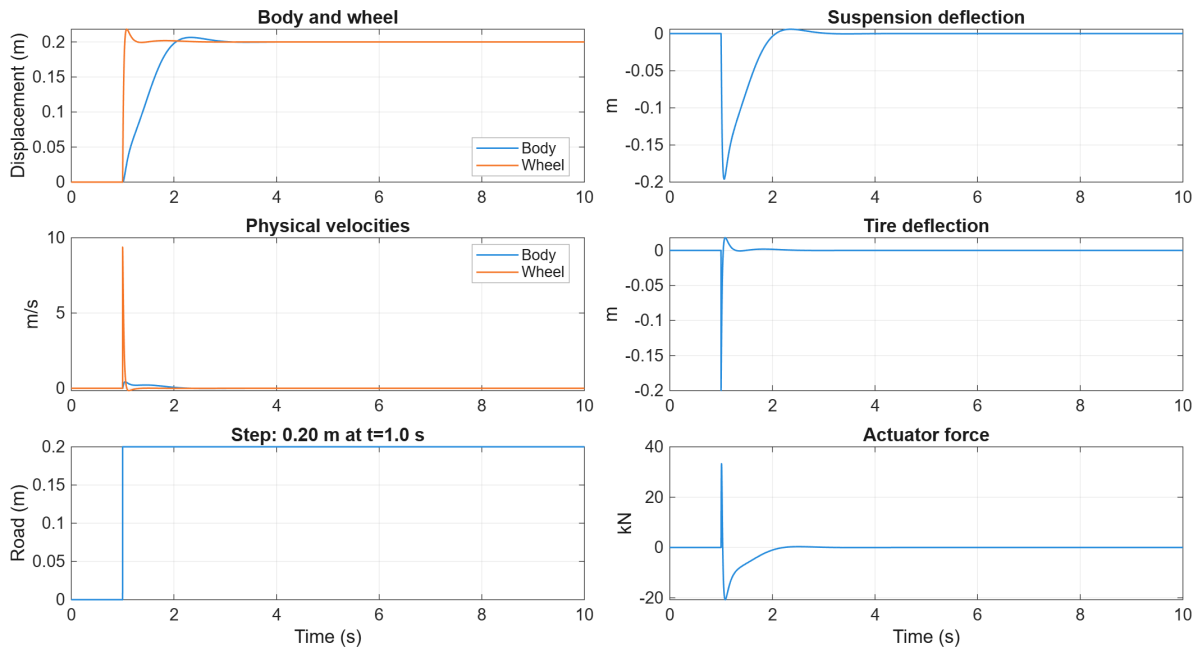


Figure 3: LQR step response with the documented five-state gain and weights.

Corrected MATLAB figures: sinusoidal road

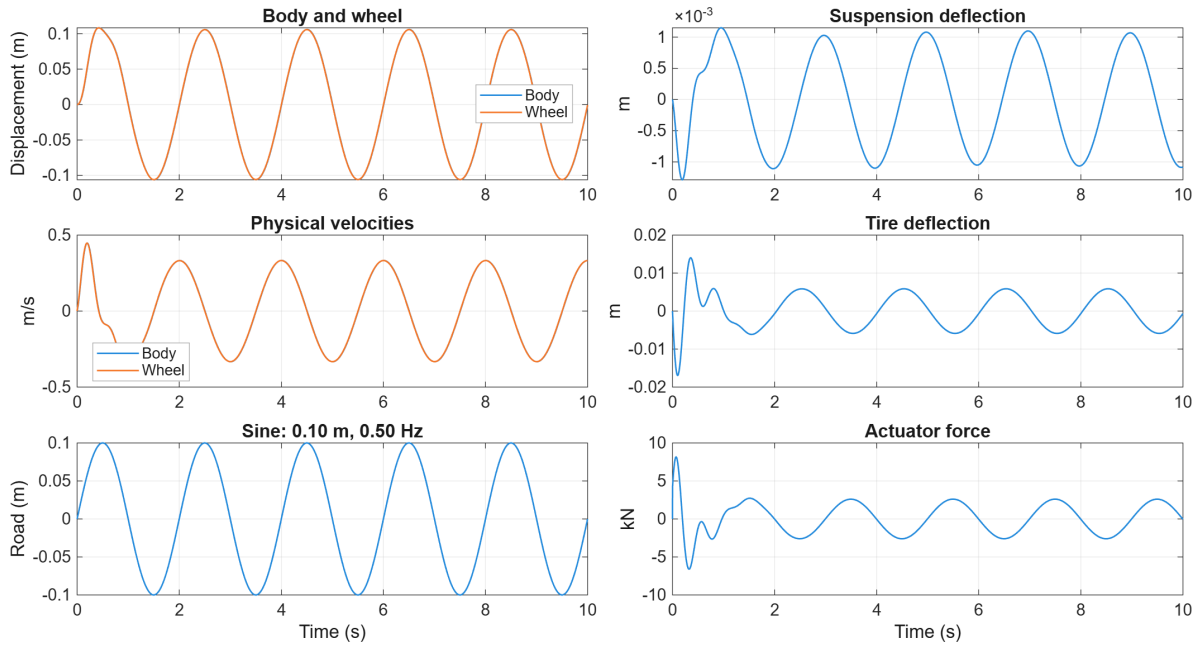


Figure 4: PID response to the 0.10 m, 0.5 Hz sine, including the startup transient.

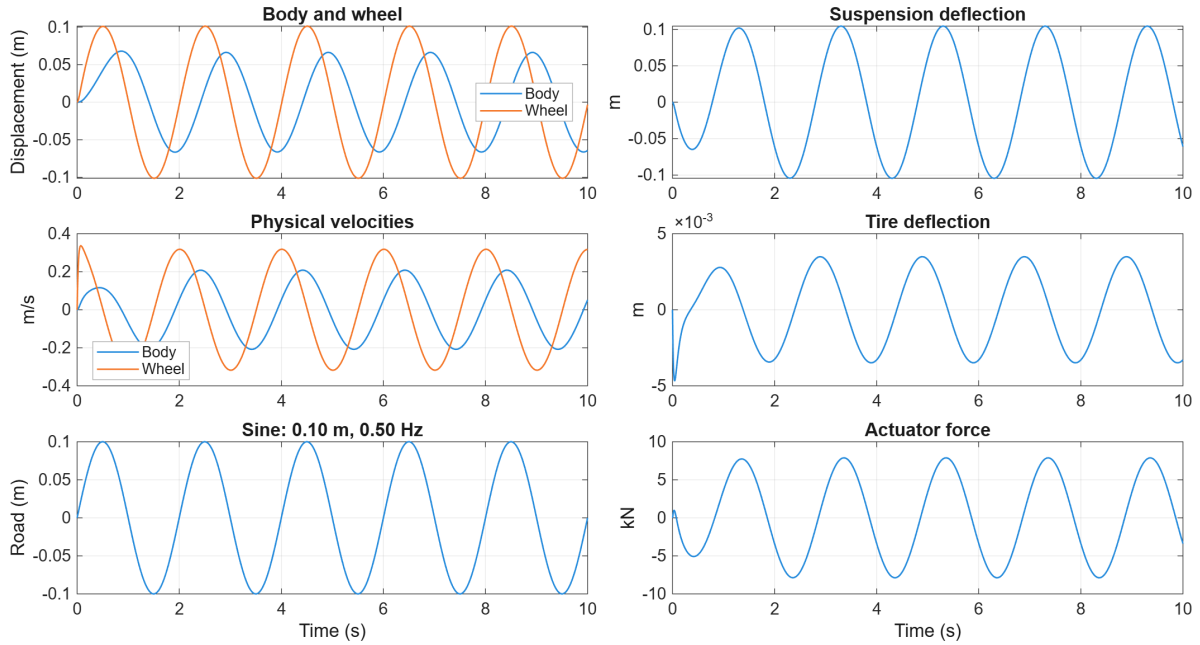


Figure 5: LQR response to the same sine. Improved comfort comes with substantial travel and force.

Passive baseline in MATLAB

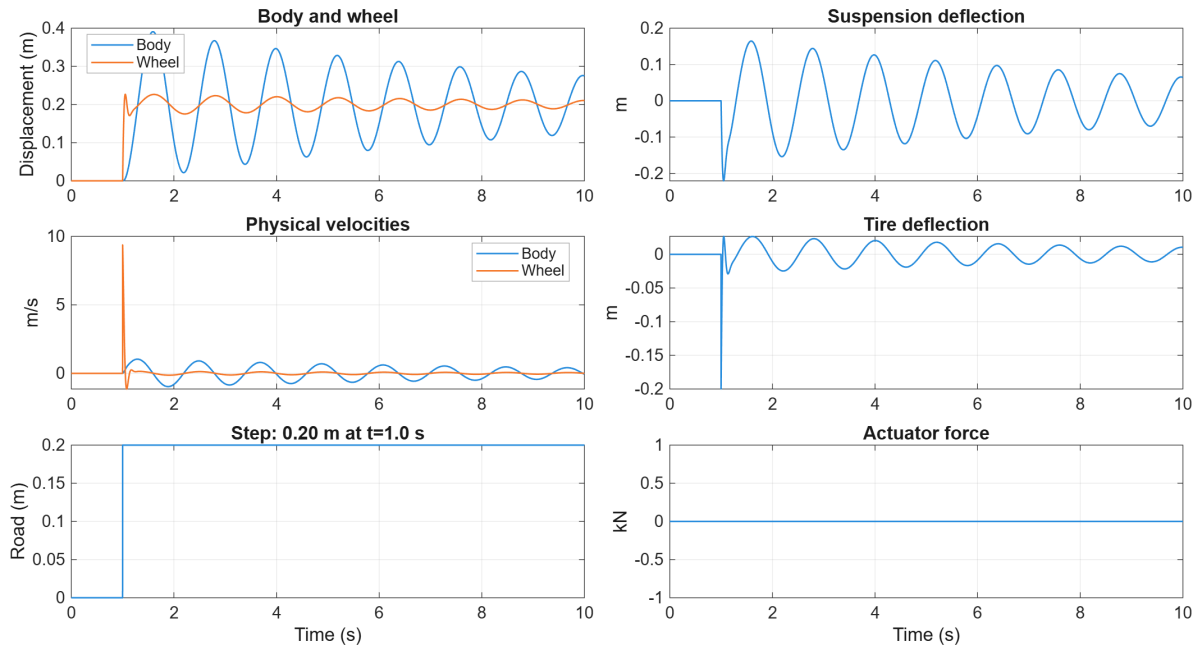


Figure 6: Passive response to the road step. Slow body-mode decay persists beyond the record.

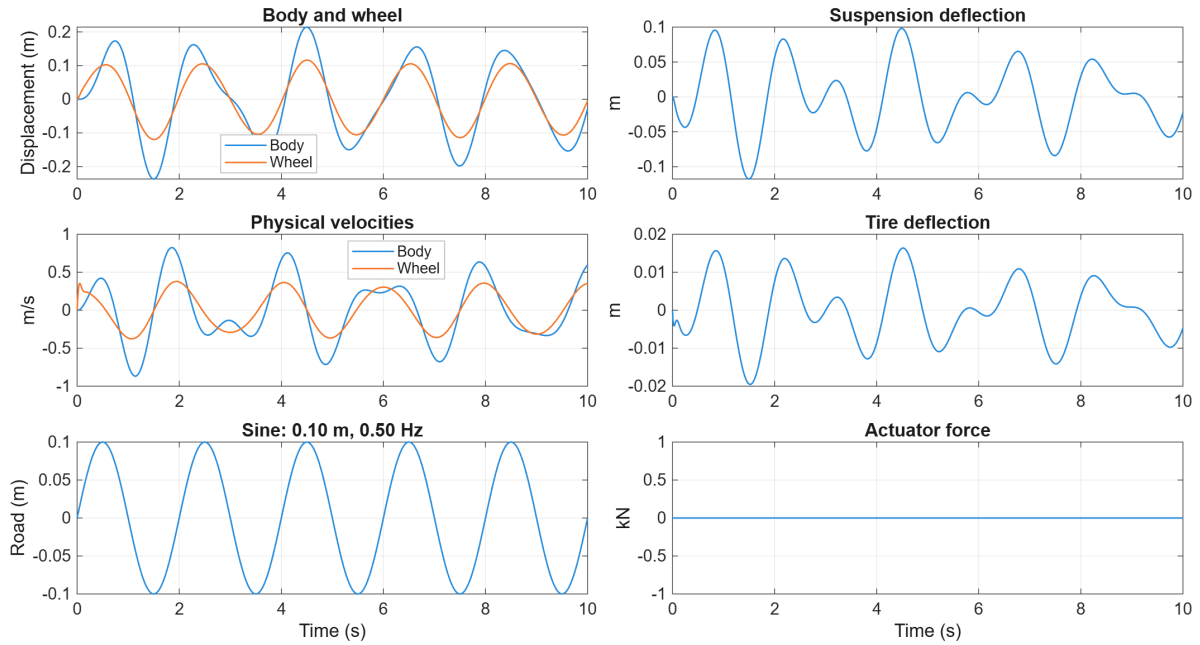


Figure 7: Passive sine response used as the common baseline for both controllers.

Task 3(a): Comfort and road-holding tradeoffs

In the step case LQR reduces acceleration RMS from 2.498 to 0.753 m/s² and tire RMS from 0.01364 to 0.00613 m relative to passive operation. However, peak travel remains about 0.196 m. The PID's 0.00625 m peak travel looks attractive in isolation, but its 13.275 m/s² acceleration RMS and enormous force spike show the price of tying body motion tightly to the wheel. In the sine case, LQR reduces both acceleration and tire RMS more than this PID tuning, while allowing about 0.104 m peak travel and using 5.384 kN RMS force. No controller dominates all metrics. Tire deflection is only a proxy: contact requires examining normal force and a model that permits tire separation.

Task 3(b): Practical actuator and implementation limits

The ideal PID step peak is about 7.81 MN, compared with the LQR peak of 33.3 kN. Neither value should be accepted as a hardware requirement without a realistic road transition and actuator model. Required stroke must accommodate suspension travel, and speed must accommodate $\dot{x}_s$. The actuator's mechanical power is $P_{\text{mech}} = u(\dot{z}_s - \dot{z}_u) = u\dot{x}_s$, not u^2 ; electrical energy also depends on losses and regeneration. RMS force is useful for effort comparison but is not an energy estimate.

Replace ideal derivative action with, for example,

$$C_{\text{PID},f}(p) = K_p + K_i/p + \frac{K_d p}{1 + \tau_f p}, \quad \tau_f > 0. \quad (60)$$

Introduce the filter state, retune, and recheck poles and disturbance responses. Add force/rate saturation and anti-windup. Integral-augmented LQR also needs anti-windup if the actuator saturates; its optimality and linear stability results do not automatically carry over to saturation. An implementation also needs sensors or an observer for the physical/transformed states; the present solution assumes full-state availability and known road input where required for reconstruction.

Task 3(c): What can be concluded about robustness?

For these nominal parameters and two specified roads, LQR provides better comfort and tire-deflection metrics than the supplied PID, but requires more travel and, for the sine, more RMS force. This is evidence about nominal performance, not proof of greater robustness to uncertain mass, damping, sensor noise, or delays.

A defensible next study would hold each gain fixed, vary M_s (for example, $\pm 20\%$) and damping ($\pm 20\%$), sweep road frequency, and recompute stability, RMS, travel, and force. If transformed-state feedback is reconstructed with nominal parameters, retain that actual estimator/reconstruction rather than giving the controller exact uncertain states. Add measurement noise, delay, and actuator limits, and report the worst cases and uncertainty ranges. Those additional tests are not claimed by the six nominal cases in this solution.

Task 3(d): Data-driven adaptation and preview

Road-preview sensors could provide feedforward action before a bump arrives. Gain scheduling could adapt to load and speed; data-driven optimization could choose validated tuning sets from road statistics. Required data include synchronized road height, speed, body/wheel motion, forces, and representative loads. Preview latency, occlusion, unseen roads, sensor faults, and changing tire properties must be tested. Constrain adaptation to a validated operating region, monitor force/travel, and retain a stable fallback controller. Prediction quality alone is insufficient evidence of closed-loop safety or reliability.

Appendix A: MATLAB implementation

The following are the actual executable repository files, included directly with \lstinputlisting to keep the printed code synchronized. They retain the original MATLAB listing colors, line numbers, light-gray background, and frame. Run `quarter_car_demo_solution.m` from MATLAB. Change `control_mode` and `road_input` to select a case. The separate `animate_step_comparison.m` produces controlled/passive overlay animations.

Configuration

```
{
  "Ms": 2500, "Mu": 320, "Ks": 80000, "Cs": 350,
  "Kt": 500000, "Ct": 15020,
  "duration": 10, "dt": 0.001,
  "step_amplitude": 0.2, "step_time": 1,
  "sine_amplitude": 0.1, "sine_frequency": 0.5,
  "Q": [30000000, 80000000, 80000000, 20000000, 200000000000],
  "R": 0.01,
  "Kp": 208000, "Kd": 832000, "Ki": 624000
}
```

Main script: parameters, simulation, plots, and animation

```
1  % ME 4030: run this script from any working directory.
2  clear; close all; clc;
3  here = fileparts(mfilename('fullpath'));
4  addpath(here, fullfile(here, '..', 'shared'));
5  control_mode = 'LQR'; % 'NONE', 'PID', or 'LQR'
6  road_input = 'step'; % 'step' or 'sine'
7  show_animation = true;
8  r = simulate_quarter_car(control_mode, road_input);
9  figure('Color','w','Name',[control_mode ' / ' road_input], ...
10     'Position',[100 100 1100 800]);
11  tiledlayout(3,2,'TileSpacing','compact');
12  nexttile; plot(r.t,[r.zs r.zu]); grid on;
13  title('Body and wheel'); ylabel('Displacement (m)'); legend('Body','Wheel');
14  nexttile; plot(r.t,r.zs-r.zu); grid on;
15  title('Suspension deflection'); ylabel('m');
16  nexttile; plot(r.t,[r.zs_dot r.zu_dot]); grid on;
17  title('Physical velocities'); ylabel('m/s'); legend('Body','Wheel');
18  nexttile; plot(r.t,r.zu-r.zr); grid on;
19  title('Tire deflection'); ylabel('m');
20  nexttile; plot(r.t,r.zr); grid on;
21  title(r.label); ylabel('Road (m)'); xlabel('Time (s)');
22  nexttile; plot(r.t,r.u/1000); grid on;
23  title('Actuator force'); ylabel('kN'); xlabel('Time (s)');
24  style_quarter_car_figure(gcf);
25  metrics = quarter_car_performance_eval(r);
26  disp(struct2table(metrics));
27  if show_animation
28      opts = struct('carSpeed',1);
29      uinfo = struct('t',r.t,'zr',r.zr,'label',r.label,'u',r.u, ...
30          'zs_dot',r.zs_dot,'zu_dot',r.zu_dot);
31      animate_quarter_car(r.t,r.zs,r.zu,r.zr,opts,uinfo);
32  end
```

Model, PID/LQR control, and exact simulation

```

1 function result = simulate_quarter_car(control_mode, road_input)
2 %SIMULATE_QUARTER_CAR Reference linear active-suspension simulation.
3 % result = simulate_quarter_car('LQR', 'step');
4 % Modes: NONE, PID, LQR. Inputs: step, sine. SI units throughout.
5 if nargin < 1, control_mode = 'LQR'; end
6 if nargin < 2, road_input = 'step'; end
7 p = jsondecode(fileread(fullfile(fileparts(mfilename('fullpath'))), 'config.json'));
8 Ms=p.Ms; Mu=p.Mu; Ks=p.Ks; Cs=p.Cs; Kt=p.Kt; Ct=p.Ct;
9 alpha = Cs/Ms + Cs/Mu + Ct/Mu;
10 % x = [zs; zs_dot; s; eta], s = zs-zu.
11 % eta = s_dot - (Ct/Mu)*zs + alpha*s + (Ct/Mu)*zr.
12 A = [0, 1, 0, 0;
13      -Cs*Ct/(Ms*Mu), 0, Cs/Ms*alpha-Ks/Ms, -Cs/Ms;
14      Ct/Mu, 0, -alpha, 1;
15      Kt/Mu, 0, -(Ks/Ms+Ks/Mu+Kt/Mu), 0];
16 Bu = [0; 1/Ms; 0; 1/Ms+1/Mu];
17 Bw = [0; Cs*Ct/(Ms*Mu); -Ct/Mu; -Kt/Mu];
18 C = [0 0 1 0];
19 Aa = [A zeros(4,1); C 0];
20 Bua = [Bu; 0]; Bwa = [Bw; 0];
21 % u = -K*xa + F*zr. PID differentiates s, not eta.
22 F = 0;
23 switch upper(control_mode)
24     case 'LQR'
25         K = lqr(Aa, Bua, diag(p.Q), p.R);
26     case 'PID'
27         K = [p.Kp*C + p.Kd*(C*A), p.Ki];
28         F = -p.Kd*(C*Bw);
29     case 'NONE'
30         K = zeros(1,5);
31     otherwise
32         error('Choose NONE, PID, or LQR.');

```

```

70     G=E.*V(1:n,n+1:end);
71     energies(j)=sum(sum((z(1:end-1,:)*G).*(z(1:end-1,:))));
72 end
73 x = xa(:,1:4);
74 s_dot = x*A(3,:).' + Bw(3)*zr;
75 u = -xa*K.' + F*zr;
76 zs = x(:,1); zu = zs-x(:,3);
77 zs_dot = x(:,2); zu_dot = zs_dot-s_dot;
78 zs_ddot = (-Ks*x(:,3)-Cs*s_dot+u)/Ms;
79 result = struct('t',t,'zr',zr,'zs',zs,'zu',zu,'zs_dot',zs_dot, ...
80     'zu_dot',zu_dot,'s_dot',s_dot,'zs_ddot',zs_ddot,'u',u, ...
81     'xa',xa,'K',K,'F',F,'poles',eig(Acl),'mode',upper(control_mode), ...
82     'road',lower(road_input),'label',label,'parameters',p,'energies',energies);
83 end

```

Performance metrics

```

1 function metrics = quarter_car_performance_eval(r)
2 % Whole-record RMS metrics; step transients measured from disturbance onset.
3 % NaN marks metrics not applicable to sine inputs; Inf means not settled.
4 t = r.t; T = t(end)-t(1);
5 rms_exact = sqrt(max(r.energies,0)/T);
6 metrics = struct('acceleration_rms_m_s2',rms_exact(1), ...
7     'tire_deflection_rms_m',rms_exact(2), ...
8     'max_suspension_deflection_m',max(abs(r.zs-r.zu)), ...
9     'force_rms_kN',rms_exact(3)/1000, ...
10    'force_peak_kN',max(abs(r.u))/1000, ...
11    'body_overshoot_percent',NaN,'body_settling_time_s',NaN,'suspension_recovery_time_s',NaN);
12 if strcmpi(r.road,'step')
13     target = r.parameters.step_amplitude;
14     post = find(t >= r.parameters.step_time);
15     y = r.zs(post);
16     metrics.body_overshoot_percent = max(0,100*max(sign(target)*(y-target))/abs(target));
17     outside = find(abs(y-target)>0.02*abs(target),1,'last');
18     if isempty(outside)
19         metrics.body_settling_time_s = 0;
20     elseif outside == numel(post)
21         metrics.body_settling_time_s = Inf;
22     else
23         metrics.body_settling_time_s = t(post(outside+1))-r.parameters.step_time;
24     end
25     outside = find(abs(r.zs(post)-r.zu(post))>0.02*abs(target),1,'last');
26     if isempty(outside)
27         metrics.suspension_recovery_time_s=0;
28     elseif outside==numel(post)
29         metrics.suspension_recovery_time_s=Inf;
30     else
31         metrics.suspension_recovery_time_s=t(post(outside+1))-r.parameters.step_time;
32     end
33 end
34 end

```

Appendix B: Reproduction and checks

Run `tools/validate_matlab.m` for the native six-case checks and `tools/export_document_figures.m` to regenerate the plots in this PDF. For independent Python checks and the document build, run:

```

python tools/build_previews.py
python tools/validate_repository.py
python tools/build_documents.py

```

The default LaTeX engine is Tectonic in the Conda environment `latex-env`.